

UNIVERSIDAD CATÓLICA DEL URUGUAY

Campus Salto

Sistema de Voto Electrónico

Trabajo Obligatorio — Bases de Datos 2

Integrantes:

Adolfo Bravo

Agustín Cigaran

Brahian Núñez

Docente: Jorge Martínez

Curso: Bases de Datos 2 | Año: 2026

Fecha de entrega: 2026

Índice

1. Introducción
2. Análisis del Contexto
 - 2.1 Entidades detectadas
 - 2.2 Atributos de cada entidad
 - 2.3 Relaciones entre entidades
 - 2.4 Reglas de negocio y restricciones
3. Solución al desafío del voto secreto
4. Metodología
5. Modelo Entidad-Relación (MER)
6. Modelo Físico — DDL
 - 6.1 Script DDL completo
 - 6.2 Script DML (datos de prueba)
7. Aplicación
 - 7.1 Arquitectura
 - 7.2 Backend — API REST (Node.js / Express)
 - 7.3 Frontend — Interfaz web
 - 7.4 Docker y despliegue
8. Consultas y reportes (QDD)
9. Conclusiones
10. Referencias

1. Introducción

El presente trabajo fue desarrollado en el marco del curso Bases de Datos 2 de la Universidad Católica del Uruguay, Campus Salto, bajo la docencia del Prof. Jorge Martínez. El objetivo del trabajo fue diseñar e implementar un prototipo de sistema de voto electrónico para la Corte Electoral del Uruguay, tomando como contexto las elecciones del 11 de mayo de 2025.

El sistema abarca desde el modelado conceptual de la base de datos —mediante un Diagrama Entidad-Relación— hasta la implementación de una aplicación cliente-servidor funcional, con persistencia en MySQL, lógica de negocio en Node.js y una interfaz web que permite consultar los datos electorales.

El principal desafío del trabajo radicó en garantizar dos propiedades contradictorias: registrar que cada ciudadano ejerció su derecho al voto (obligatoriedad) sin que sea posible rastrear a quién votó (secreto del sufragio). La solución adoptada se detalla en la sección 3.

2. Análisis del Contexto

A continuación se describe el relevamiento de requerimientos a partir de la consigna, identificando entidades, atributos, relaciones y restricciones del dominio electoral uruguayo.

2.1 Entidades detectadas

Del análisis del enunciado se identificaron las siguientes entidades principales:

- Ciudadano — toda persona habilitada para votar (mayor de 18 años con Credencial Cívica).
- Candidato — ciudadano que se postula a un cargo electivo en representación de un partido.
- Partido político — organización que presenta candidatos y listas.
- Elección — evento electoral (presidencial, municipal, referéndum, plebiscito, ballotage).
- Tipo de elección — categoría que clasifica cada elección.
- Papeleta — hoja de votación; puede ser una lista o una consulta (SÍ/NO).
- Lista — subtipo de papeleta asociada a un partido y un órgano electoral.
- Departamento — división político-administrativa del Uruguay.
- Localidad — pueblo, ciudad o paraje dentro de un departamento.
- Establecimiento — escuela, liceo u otro edificio donde funciona un local de votación.
- Circuito — unidad básica de votación; tiene una mesa y una lista de credenciales autorizadas.
- Mesa — grupo de tres ciudadanos (presidente, secretario, vocal) que supervisan un circuito en una elección.
- Miembro de mesa — ciudadano empleado público que integra una mesa.
- Voto — acto de sufragar; registra el estado y las papeletas incluidas, pero no el votante.
- Detalle de voto — relación entre un voto y las papeletas que contiene.
- Participación electoral — registro de que un ciudadano votó en una elección (sin indicar a quién).

2.2 Atributos de cada entidad

La siguiente tabla resume las entidades con sus atributos y claves:

Entidad	Atributos principales	PK / FK / Notas
DEPARTAMENTO	id_departamento, nombre	PK: id_departamento
TIPO_ELECCION	id_tipo_eleccion, nombre, descripción	PK: id_tipo_eleccion
ELECCION	id_eleccion, nombre, fecha, estado	PK: id_eleccion FK: id_tipo_eleccion
PARTIDO	id_partido, nombre, dirección_sede, presidente, vicepresidente	PK: id_partido
CIUDADANO	id_ciudadano, ci, credencial_cívica, nombre_completo, fecha_nacimiento, tiene_credencial	PK: id_ciudadano CC y CI únicos
CANDIDATO	id_candidato, ci, credencial_cívica, nombre_completo	PK: id_candidato FK: id_partido

Entidad	Atributos principales	PK / FK / Notas
LOCALIDAD	id_localidad, nombre, tipo_localidad	PK: id_localidad FK: id_departamento
ESTABLECIMIENTO	id_establecimiento, nombre, tipo, dirección	PK: id_establecimiento FK: id_localidad
CIRCUITO	id_circuito, numero_circuito, barrio_zona, accesible, estado, votos_emitidos	PK: id_circuito FK: id_establecimiento
MESA	id_mesa, fecha_apertura, fecha_cierre, estado	PK: id_mesa FK: id_circuito, id_eleccion
MIEMBRO_MESA	id_miembro_mesa, rol_mesa, organismo_estado	PK: id_miembro_mesa FK: id_ciudadano, id_mesa
PAPELETA	id_papeleta, tipo_papeleta, color, opcion_consulta	PK: id_papeleta FK: id_eleccion
LISTA	id_lista, numero_lista	PK/FK: id_lista hereda de PAPELETA FK: id_partido, id_candidato_principal, id_departamento
INTEGRACION_LISTA	orden_aparicion	PK: (id_lista, id_candidato) FK: ambas
VOTO	id_voto, fecha_hora, estado_voto, observado	PK: id_voto FK: id_mesa — SIN vínculo a ciudadano
DETALLE_VOTO	(relación)	PK: (id_voto, id_papeleta) registra qué papeletas tiene el voto
PARTICIPACION_ELECTORAL	id_participacion, fecha_hora, observado	PK: id_participacion FK: id_ciudadano, id_eleccion — certifica que la persona votó

2.3 Relaciones entre entidades

- Un DEPARTAMENTO tiene muchas LOCALIDADES (1:N).
- Una LOCALIDAD tiene muchos ESTABLECIMIENTOS (1:N).
- Un ESTABLECIMIENTO tiene muchos CIRCUITOS (1:N).
- Un CIRCUITO tiene una MESA por ELECCION (1:1 por elección).
- Una MESA tiene tres MIEMBROS_MESA: presidente, secretario y vocal (1:3 exactamente).
- Una ELECCION tiene muchas PAPELETAS (1:N); una PAPELETA pertenece a una sola ELECCION.
- Una LISTA es un subtipo de PAPELETA (herencia/especialización: LISTA hereda id_papeleta).
- Una LISTA pertenece a un PARTIDO (N:1) y tiene muchos CANDIDATOS a través de INTEGRACION_LISTA (N:M).

- Un VOTO se emite en una MESA (N:1) y referencia muchas PAPELETAS a través de DETALLE_VOTO (N:M).
- Una PARTICIPACION_ELECTORAL vincula un CIUDADANO a una ELECCION, registrando únicamente el hecho de haber votado (N:M descompuesto).

2.4 Reglas de negocio y restricciones

#	Regla	Implementación en el modelo
1	Voto secreto	VOTO no tiene FK a CIUDADANO. La participación se registra en PARTICIPACION_ELECTORAL (tabla separada).
2	Un voto por ciudadano por elección	PARTICIPACION_ELECTORAL tiene PK única (id_ciudadano, id_eleccion). Se puede agregar UNIQUE constraint.
3	Mesa se cierra y no vuelve a abrir	Campo estado en MESA solo acepta 'Abierta' o 'Cerrada'. La lógica en la API impide reapertura.
4	No se muestran resultados antes de cierre	El endpoint de resultados verifica estado = 'Cerrada' antes de devolver datos.
5	Voto observado si el circuito no corresponde	Campo observado en VOTO y PARTICIPACION_ELECTORAL. Requiere autorización del presidente de mesa.
6	Listas pertenecen a un partido político	LISTA tiene FK id_partido a PARTIDO.
7	Varios tipos de elección en simultáneo	ELECCION es independiente; MESA referencia id_eleccion; el VOTO puede tener múltiples DETALLE_VOTO.
8	Estado del voto: válido / nulo / en blanco	Campo estado_voto en VOTO (VARCHAR).

3. Solución al desafío del voto secreto

La consigna plantea una tensión fundamental: el voto es obligatorio (se debe saber si alguien votó) y secreto (no se puede saber a quién votó). Esta sección explica la solución adoptada en el modelo.

El problema

Si la tabla VOTO tuviera una clave foránea a CIUDADANO, sería trivialmente posible consultar a quién votó cada persona. Por lo tanto, esa relación directa queda prohibida por diseño.

La solución: dos tablas separadas

Se utilizan dos tablas con responsabilidades distintas y sin vínculo entre sí:

- PARTICIPACION_ELECTORAL — registra únicamente que el ciudadano X votó en la elección Y, con fecha y hora. No contiene ninguna referencia al voto ni a la papeleta elegida.
- VOTO — registra el contenido del voto (qué papeletas incluye, si es válido o nulo, si fue observado). No contiene ninguna referencia al ciudadano.

De esta forma, aunque se accediera a ambas tablas, no existe ninguna clave que las vincule. Es imposible —a nivel de base de datos— saber qué votó un ciudadano específico.

Flujo en el sistema

1. El ciudadano se identifica en la aplicación (CI + credencial).
2. El sistema verifica en PARTICIPACION_ELECTORAL que no haya votado aún en esa elección.
3. El ciudadano selecciona sus papeletas y confirma.
4. Se genera un registro en VOTO (con las papeletas en DETALLE_VOTO) — sin referencia al ciudadano.
5. Se genera un registro en PARTICIPACION_ELECTORAL — sin referencia al voto.
6. El contador votos_emitidos del CIRCUITO se incrementa.

El resultado es que sólo existe correlación temporal entre ambos registros, pero no una clave relacional que los vincule.

4. Metodología

El trabajo siguió un proceso iterativo compuesto por las siguientes etapas:

7. Análisis del dominio: lectura y descomposición del enunciado en entidades, atributos y restricciones.
8. Modelado conceptual: confección del Diagrama Entidad-Relación (MER) usando draw.io.
9. Diseño del modelo físico: traducción del MER al DDL de MySQL 8.0, respetando tipos de datos, claves primarias, claves foráneas y restricciones de unicidad.
10. Carga de datos de prueba: redacción del DML con datos representativos del contexto electoral uruguayo (departamentos reales, partidos actuales, elecciones 2024-2025).
11. Implementación de la API: desarrollo de un servidor REST en Node.js con Express y el driver mysql2, exponiendo los endpoints necesarios para consultar la base de datos.
12. Implementación del frontend: interfaz web en HTML/CSS/JS vanilla que consume la API y presenta los datos en tablas interactivas.
13. Contenerización: empaquetado de los tres componentes (MySQL, API, Adminer) con Docker Compose para facilitar el despliegue reproducible.
14. Verificación: prueba de cada funcionalidad requerida y revisión del cumplimiento de las reglas de negocio.

5. Modelo Entidad-Relación (MER)

El diagrama MER fue confeccionado en draw.io y se encuentra en la carpeta Diagramas/ del repositorio (archivo MER.drawio.png). A continuación se describe su estructura general.

Núcleo del modelo

El modelo se organiza alrededor de tres ejes principales:

- Eje geográfico: DEPARTAMENTO → LOCALIDAD → ESTABLECIMIENTO → CIRCUITO → MESA. Esta cadena refleja la jerarquía de ubicación del voto.
- Eje electoral: ELECCION → PAPELETA → LISTA → INTEGRACION_LISTA → CANDIDATO. Esta cadena describe qué se vota en cada evento.
- Eje del sufragio: MESA → VOTO → DETALLE_VOTO ↔ PAPELETA, y paralelamente CIUDADANO → PARTICIPACION_ELECTORAL → ELECCION. Esta dualidad garantiza el secreto del voto.

Generalización / Especialización

La entidad LISTA es una especialización de PAPELETA. Hereda su clave primaria (id_papeleta = id_lista) y agrega los atributos propios de una lista política: número de lista, partido, candidato principal, órgano y departamento. Las papeletas de tipo 'Consulta' (referéndum, plebiscito) no son LISTA; solo son PAPELETA con los campos opcion_consulta y color.

Extensibilidad

El modelo fue diseñado para soportar todos los tipos de elección definidos en la consigna. La tabla TIPO_ELECCION permite agregar nuevas categorías sin modificar el esquema. Las tablas ELECCION y PAPELETA permiten modelar elecciones simultáneas (por ejemplo, presidenciales + plebiscito en la misma fecha). El campo estado en MESA y ELECCION habilita el control del ciclo de vida de cada evento.

6. Modelo Físico — DDL

El modelo físico se implementó en MySQL 8.0 con charset utf8mb4 para soporte completo de caracteres especiales del español (tildes, eñes). A continuación se presentan las sentencias más relevantes del script DDL, con comentarios explicativos.

6.1 Tablas principales y solución del voto secreto

Las siguientes tablas son el núcleo de la solución al voto secreto. Nótese que VOTO no tiene ninguna FK hacia CIUDADANO, mientras que PARTICIPACION_ELECTORAL no tiene FK hacia VOTO:

```
CREATE TABLE CIUDADANO (
  id_ciudadano INT AUTO_INCREMENT PRIMARY KEY,
  ci VARCHAR(20) NOT NULL UNIQUE,
  credencial_civica VARCHAR(20) NOT NULL UNIQUE,
  nombre_completo VARCHAR(150) NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  tiene_credencial BOOLEAN DEFAULT TRUE
);

CREATE TABLE ELECCION (
  id_eleccion INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  fecha DATE NOT NULL,
  estado VARCHAR(50),
  id_tipo_eleccion INT,
  FOREIGN KEY (id_tipo_eleccion) REFERENCES TIPO_ELECCION(id_tipo_eleccion)
);

CREATE TABLE VOTO (
  id_voto INT AUTO_INCREMENT PRIMARY KEY,
  fecha_hora DATETIME NOT NULL,
  estado_voto VARCHAR(50) NOT NULL, -- 'Válido', 'Nulo', 'Blanco', 'Observado'
  observado BOOLEAN DEFAULT FALSE,
  id_mesa INT,
  FOREIGN KEY (id_mesa) REFERENCES MESA(id_mesa)
  -- ⚠ SIN FK a CIUDADANO: voto secreto garantizado
);

CREATE TABLE PARTICIPACION_ELECTORAL (
  id_participacion INT AUTO_INCREMENT PRIMARY KEY,
  fecha_hora DATETIME NOT NULL,
  observado BOOLEAN DEFAULT FALSE,
  id_ciudadano INT,
  id_eleccion INT,
  FOREIGN KEY (id_ciudadano) REFERENCES CIUDADANO(id_ciudadano),
  FOREIGN KEY (id_eleccion) REFERENCES ELECCION(id_eleccion)
  -- Registra QUE votó, pero no A QUIÉN
);

CREATE TABLE DETALLE_VOTO (
```

```
id_voto    INT,  
id_papeleta INT,  
PRIMARY KEY (id_voto, id_papeleta),  
FOREIGN KEY (id_voto) REFERENCES VOTO(id_voto),  
FOREIGN KEY (id_papeleta) REFERENCES PAPELETA(id_papeleta)  
);
```

6.2 Decisiones de diseño en el DDL

- Herencia de LISTA: la tabla LISTA usa su id_lista como PK y además FK a PAPELETA(id_papeleta). Esto implementa la herencia de tabla por clase (Table-Per-Class).
- Estado de la mesa: el campo estado en MESA acepta solo 'Abierta' o 'Cerrada'. La restricción de no reapertura se impone en la lógica de la API.
- Votos emitidos: el campo votos_emitidos en CIRCUITO es un contador desnormalizado que se incrementa con cada voto, permitiendo reportes rápidos sin hacer COUNT sobre la tabla VOTO.
- Charset: el Dockerfile de MySQL configura utf8mb4_unicode_ci para evitar problemas con caracteres latinos.

6.3 Script DML — Datos de prueba

El archivo Bd/02_dml.sql carga datos representativos que permiten probar el sistema sin necesidad de ingresar datos manualmente. Incluye:

- 8 departamentos reales del Uruguay.
- 5 partidos políticos actuales (Frente Amplio, Partido Nacional, Partido Colorado, Partido Independiente, Cabildo Abierto).
- 20 ciudadanos ficticios con CI y credencial cívica únicos.
- 10 candidatos con datos inspirados en figuras políticas reales.
- 5 tipos de elección y 5 elecciones (incluyendo las del 27/10/2024 y 11/05/2025).
- 10 establecimientos (escuelas y liceos) distribuidos en distintos departamentos.
- 11 circuitos con su estado y contador de votos.
- 9 papeletas de distintos tipos: listas, consultas SÍ/NO.
- 20 votos con distintos estados (válido, nulo, blanco, observado) y sus detalles.

7. Aplicación

7.1 Arquitectura

La aplicación sigue una arquitectura cliente-servidor de tres capas:

- Capa de datos: MySQL 8.0 ejecutándose en un contenedor Docker. Expone el puerto 3307 al host.
- Capa de lógica / API: servidor Node.js con Express que acepta peticiones HTTP REST, ejecuta consultas SQL mediante el driver mysql2 (con pool de conexiones) y devuelve JSON. Expone el puerto 3000.
- Capa de presentación: página HTML/CSS/JS servida por el propio servidor Express (carpeta public/). El navegador del usuario consume la API REST mediante fetch().
- Herramienta de administración: Adminer, una interfaz web para inspeccionar y gestionar la base de datos directamente. Expone el puerto 8080.

Componente	Tecnología	Rol en el sistema
Base de datos	MySQL 8.0	Motor relacional que almacena toda la información electoral.
Backend / API	Node.js + Express	Servidor REST que expone los endpoints y ejecuta las consultas SQL.
Frontend	HTML5 + CSS3 + JS vanilla	Interfaz web sencilla que consume la API y muestra los datos en tablas.
Contenedores	Docker + Docker Compose	Orquesta los tres servicios: MySQL, API y Adminer.
Administración BD	Adminer (puerto 8080)	Herramienta web para inspeccionar la base de datos en desarrollo.

7.2 Backend — API REST (Node.js / Express)

El servidor (App/server.js) implementa los siguientes endpoints:

Método	Endpoint	Descripción
GET	/api/ciudadanos	Lista todos los ciudadanos registrados con su habilitación electoral.
GET	/api/elecciones	Lista elecciones con tipo, fecha, estado y total de participaciones.
GET	/api/votos-partido	Votos válidos agrupados por partido y elección.
GET	/api/mesas	Estado de mesas y circuitos con votos emitidos.
GET	/api/candidatos	Candidatos con su lista, partido, orden y elección.

Cada endpoint sigue el mismo patrón: obtiene la conexión del pool, ejecuta la consulta SQL con parámetros preparados, y devuelve el resultado como JSON. Los errores se capturan con try/catch y se retornan con código HTTP 500.

El endpoint `/api/votos-partido` es el más relevante desde el punto de vista electoral: una `DETALLE_VOTO` → `PAPELETA` → `LISTA` → `PARTIDO` → `ELECCION`, filtrando únicamente los votos válidos y agrupando por partido y elección. Esto responde directamente al reporte solicitado en la consigna.

7.3 Frontend — Interfaz web

La interfaz (`App/public/index.html`) es una Single Page Application ligera implementada con HTML5, CSS3 y JavaScript vanilla. Sus características principales son:

- Diseño oscuro (dark mode) con variables CSS para una apariencia profesional y coherente.
- Navegación lateral con cinco secciones: Ciudadanos, Elecciones, Votos por partido, Mesas y circuitos, Candidatos y listas.
- Carga bajo demanda: los datos se solicitan a la API únicamente cuando el usuario presiona el botón de consulta, evitando peticiones innecesarias.
- Tabla dinámica: el resultado de cada endpoint se renderiza en una tabla HTML generada con JavaScript, con formato condicional (pastillas de colores para estados, fechas localizadas en español Uruguay, números con tipografía tabular).
- Indicador de estado de conexión: muestra en tiempo real si la API está disponible (verde: conectado, rojo: sin conexión).
- Contador de registros: badge dinámico que muestra el número de filas devueltas por cada consulta.

7.4 Docker y despliegue

El proyecto se despliega completamente con un único comando desde la raíz del repositorio:

```
docker compose up -d --build
```

El archivo `docker-compose.yml` define tres servicios:

- `mysql`: construido desde el Dockerfile raíz, que extiende la imagen oficial `mysql:8.0` y copia los scripts DDL y DML en `/docker-entrypoint-initdb.d/` para que se ejecuten automáticamente al iniciar el contenedor por primera vez.
- `api`: construida desde `App/Dockerfile`, basada en `node:20-alpine`. Copia el `package.json`, instala dependencias con `npm install`, y ejecuta `node server.js`.
- `adminer`: imagen oficial sin modificaciones, disponible en `http://localhost:8080` para administración.

La dependencia entre servicios se gestiona con `depends_on` y `healthcheck`: la API no arranca hasta que MySQL responde correctamente a `mysqladmin ping`.

8. Consultas y reportes (QDD)

La consigna solicita un bosquejo de solución usando el método QDD (Query-Driven Design). A continuación se presentan las consultas clave del sistema, mostrando la pregunta, el SQL simplificado y las tablas involucradas.

Pregunta / Consulta	SQL simplificado	Tablas involucradas
¿Cuántos votos emitió cada departamento?	SELECT d.nombre, COUNT(v.id_voto) FROM VOTO v JOIN MESA m ... JOIN DEPARTAMENTO d GROUP BY d.nombre	VOTO, MESA, CIRCUITO, ESTABLECIMIENTO, LOCALIDAD, DEPARTAMENTO
¿Cuántos votos válidos/nulos/en blanco por partido?	SELECT p.nombre, v.estado_voto, COUNT(*) FROM VOTO v JOIN DETALLE_VOTO dv JOIN LISTA l JOIN PARTIDO p GROUP BY p.nombre, v.estado_voto	VOTO, DETALLE_VOTO, PAPELETA, LISTA, PARTIDO
¿Cuántos votos obtuvo cada candidato?	SELECT c.nombre_completo, COUNT(*) FROM INTEGRACION_LISTA il JOIN LISTA l JOIN DETALLE_VOTO dv JOIN VOTO v WHERE il.orden_aparicion = 1 AND v.estado_voto = 'Válido' GROUP BY c.nombre_completo	CANDIDATO, INTEGRACION_LISTA, LISTA, DETALLE_VOTO, VOTO
¿Qué ciudadanos aún no votaron?	SELECT c.nombre_completo FROM CIUDADANO c WHERE c.id_ciudadano NOT IN (SELECT id_ciudadano FROM PARTICIPACION_ELECTORAL WHERE id_eleccion = ?)	CIUDADANO, PARTICIPACION_ELECTORA L
¿Cuántos votos observados hay por circuito?	SELECT c.numero_circuito, COUNT(*) FROM VOTO v JOIN MESA m ON ... WHERE v.observado = TRUE GROUP BY c.numero_circuito	VOTO, MESA, CIRCUITO

Las tres primeras consultas responden directamente a los reportes mínimos requeridos por la consigna: votos por departamento, votos por partido y votos por candidato, todos detallados por estado (válido, observado, anulado).

9. Conclusiones

El trabajo permitió aplicar en un contexto realista los conceptos fundamentales del diseño de bases de datos relacionales: modelado conceptual, normalización, implementación física y construcción de una aplicación que consume la base de datos.

El principal desafío —garantizar el secreto del voto sin perder el registro de participación— se resolvió de forma elegante mediante la separación en dos tablas sin vínculo directo: VOTO y PARTICIPACION_ELECTORAL. Esta solución es robusta a nivel de esquema, ya que la privacidad del votante está garantizada por diseño, no por políticas de acceso que podrían ser vulneradas.

El modelo resultante es extensible: soporta todos los tipos de elección definidos en la consigna (presidenciales, municipales, ballottage, referéndum, plebiscito) sin cambios estructurales, gracias a la tabla TIPO_ELECCION y la jerarquía PAPELETA → LISTA.

La arquitectura cliente-servidor con Docker Compose garantiza un despliegue reproducible en cualquier entorno que tenga Docker instalado, lo que facilita la evaluación y la extensión futura del prototipo.

Como trabajo a futuro se identifica la necesidad de implementar: autenticación de usuarios (votantes vs. presidentes de mesa), endpoints para registrar votos en tiempo real, validación del circuito del votante, y visualización gráfica de resultados (gráficos de barras/torta por partido).

10. Referencias

- Corte Electoral del Uruguay. (2025). Elecciones Departamentales y Municipales 2025. <https://www.corteelectoral.gub.uy>
- MySQL AB. (2024). MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- OpenJS Foundation. (2024). Node.js Documentation. <https://nodejs.org/docs/>
- Express.js. (2024). Express — Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com/>
- Docker Inc. (2024). Docker Documentation. <https://docs.docker.com/>
- Chen, P. P. (1976). The Entity-Relationship Model: Toward a Unified View of Data. ACM Transactions on Database Systems, 1(1), 9-36.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7.^a ed.). McGraw-Hill Education.
- Martínez, J. (2026). Consigna Trabajo Obligatorio — Bases de Datos 2. Universidad Católica del Uruguay, Campus Salto.